

Quick Sort vs. Merge Sort: Myth or Fact?

“Quick sort is merge sort but in place.” — That statement is **incorrect**. Although both use divide-and-conquer, their core steps, performance characteristics, and space requirements differ significantly.

1. Core Mechanism

- Quick Sort
 - Picks a pivot, partitions the array into elements less than and greater than the pivot.
 - Recursively sorts each partition in place—no merging step.
- Merge Sort
 - Splits the array in half, recursively sorts each half.
 - Merges two sorted halves into one new array, requiring extra space.

2. Time Complexity

Algorithm	Best	Average	Worst	
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	

Quick sort can degrade to $O(n^2)$ on already sorted (or adversarial) inputs unless you use randomized or median-of-three pivots.

3. Space Usage

- Quick Sort
 - In-place (broad definition): $O(\log n)$ extra space for recursion stack.
- Merge Sort
 - Out-of-place: $O(n)$ extra space to hold merged output.

4. Stability

- Quick Sort
 - Unstable by default: equal elements may change relative order.
- Merge Sort
 - Stable: preserves original order of equal elements.

Comparison Table

Property	Quick Sort	Merge Sort	
Divide Step	Partition around pivot	Split array in half	
Conquer Step	Recurse partitions	Recurse halves	
Combine Step	None	Merge two sorted halves	
Extra Space	$O(\log n)$	$O(n)$	
Stability	No	Yes	
Worst-Case Time	$O(n^2)$	$O(n \log n)$	

Bottom Line

Quick sort is **not** merge sort made in place. They share a divide-and-conquer spirit but differ in how they recombine sorted data and in their performance trade-offs.